qa-demo

Documentación del proyecto de automatización de pruebas con TypeScript · Repositorio: mariotalavera32/prueba_mario

1. Visión general

Proyecto de QA que combina varios niveles de prueba sobre sitios públicos de demostración: pruebas unitarias, E2E con Page Object Model, API, login, accesibilidad y regresión visual. Todo se ejecuta localmente y en GitHub Actions.

Herramienta	Uso
Vitest	Pruebas unitarias y cobertura (`unit/` , `src/`)
Playwright	E2E, API, login, accesibilidad y visual (`tests/`)
@axe-core/playwright	Auditoría de accesibilidad
GitHub Actions	Integración continua (`.github/workflows/qa.yml`)

2. Arquitectura

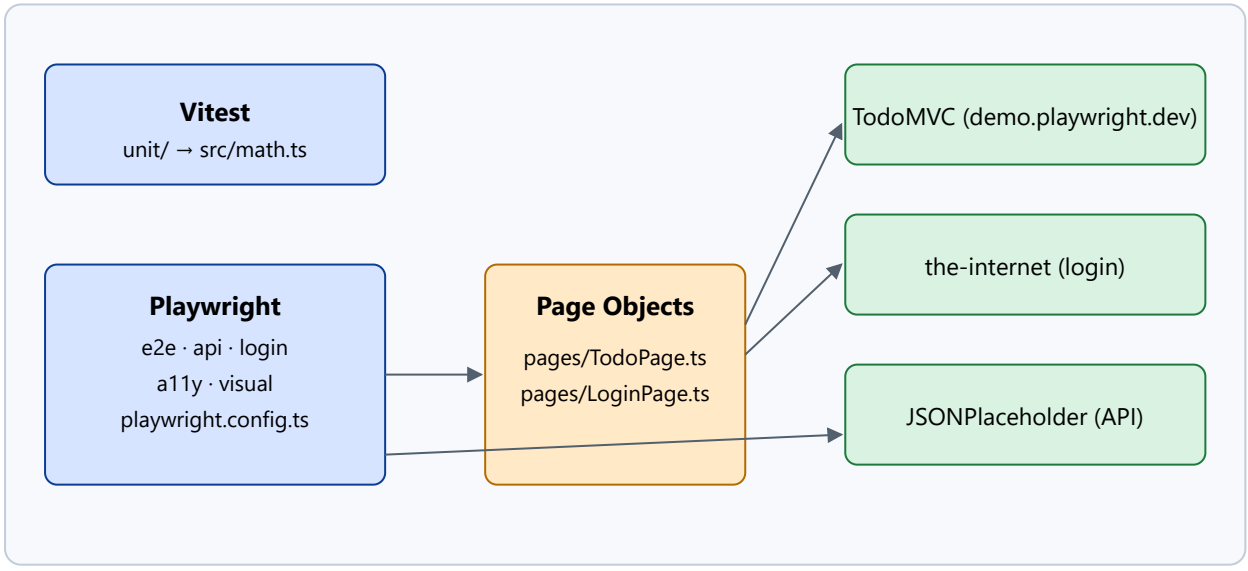


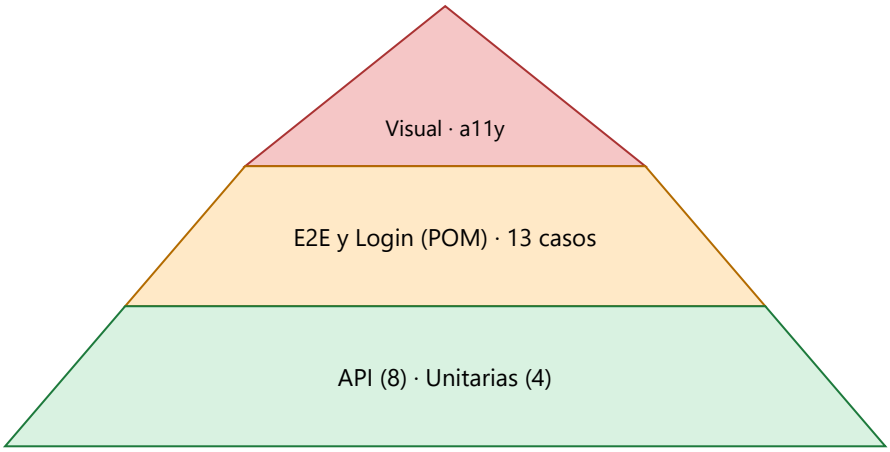
Figura 1. Componentes y sistemas bajo prueba.

Estructura de carpetas

```
qa-demo/
├─ .github/workflows/qa.yml     CI
├─ pages/                       Page Objects (TodoPage, LoginPage)
├─ src/math.ts                  Código bajo prueba unitaria
```

```
├─ unit/math.test.ts            Pruebas Vitest
├─ tests/
│  ├─ e2e.spec.ts  todo-pom.spec.ts  login.spec.ts
│  ├─ api.spec.ts  a11y.spec.ts      visual.spec.ts
│  └─ __snapshots__/              Baselines visuales
├─ playwright.config.ts
└─ README.md
```

3. Pirámide de pruebas



Más rápidas y baratas abajo · más lentas y realistas arriba

Figura 2. Distribución de las pruebas por nivel.

4. Inventario de casos

Tipo	Archivo	Casos	Detalle
Unitarias	`unit/math.test.ts`	4	Funciones de `src/math.ts`
API	`tests/api.spec.ts`	8	GET 200 (IDs 1, 50, 100), GET 404 (0, 101, 99999), POST 201, lista de 100
E2E	`tests/e2e.spec.ts`	1	Agregar y completar tarea
E2E con POM	`tests/todo-pom.spec.ts`	6	Agregar, eliminar, editar, vacías, persistencia, filtros
Login	`tests/login.spec.ts`	6	Válido, 4 inválidos parametrizados, logout
Accesibilidad	`tests/a11y.spec.ts`	1	Sin violaciones serias/críticas (axe, sin color-contrast)
Visual	`tests/visual.spec.ts`	2	Lista con tareas y estado vacío (solo Chromium)

Total: 19 casos únicos. Los casos de Playwright (15) corren en Chromium, Firefox y WebKit, salvo los visuales (solo Chromium): **75 ejecuciones** de Playwright.

5. Patrón Page Object Model

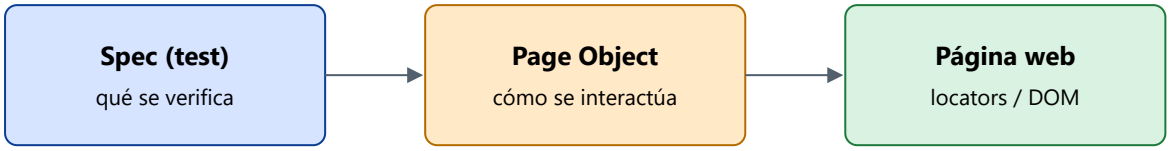


Figura 3. Los tests no conocen los selectores; solo el Page Object.

6. Integración continua

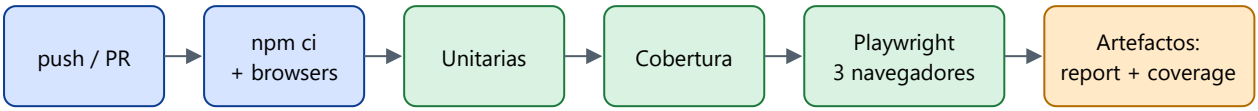


Figura 4. Pipeline de `.github/workflows/qa.yml` (Node 22, ubuntu-latest, reintentos: 2).

7. Flujo de una prueba visual

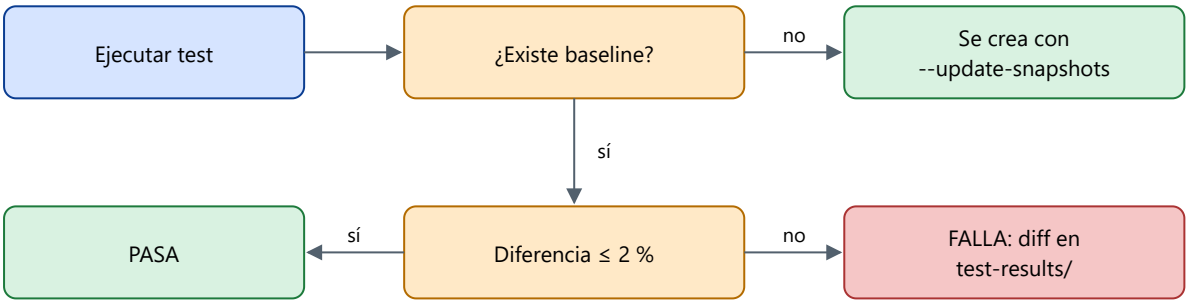


Figura 5. Decisión de una comparación visual.

Las baselines se guardan sin depender del sistema operativo (`snapshotPathTemplate`) y con tolerancia `maxDiffPixelRatio: 0.02` , para que lo generado en Windows también pase en el CI de Linux.

8. Comandos

Comando	Acción
`npm ci` y `npx playwright install chromium firefox webkit`	Instalación
`npm test`	Unitarias + E2E
`npm run test:unit` / `test:e2e` / `test:coverage`	Cada nivel por separado
`npx playwright test --project=firefox`	Un solo navegador
`npx playwright test tests/visual.spec.ts --project=chromium --update-snapshots`	Regenerar baselines
`npx playwright show-report`	Reporte HTML

9. Historial de trabajo

#	Cambio
1	Base: unitarias, E2E, API y CI
2	Login con POM, cobertura y refactor de `math`
3	Prueba de accesibilidad y paso de cobertura en CI
4	Firefox y WebKit, artefacto de cobertura, README
5	Parametrización de login y API
6	Regresión visual (Chromium)
7	Documentación de pruebas visuales y este PDF

10. Próximos pasos sugeridos

- Pruebas de rendimiento (Lighthouse).
- Reportes Allure.
- Caso visual en vista móvil y filtro "Completed".